

Estimation using COCOMO

Cocomo History

- 1981 - The original COCOMO is introduced in Dr. Barry Boehm's textbook ***Software Engineering Economics***. This model is now generally called "COCOMO 81".
- 1987 - Ada COCOMO and Incremental COCOMO are introduced (proceedings, Third COCOMO Users Group Meeting, Software Engineering Institute).
- 1988, 1989 - Refinements are made to Ada COCOMO.
- 1995, 1996 - Early papers describing COCOMO 2 published.
- 1997 - The first calibration of COCOMO II is released by Dr. Boehm, and named "COCOMO II.1997".
- 1998 - The second calibration of COCOMO II is released. It's named "COCOMO II.1998".
- 1999 - COCOMO II.1998 is renamed to COCOMO II.1999 and then to COCOMO II.2000 (all three models are identical).
- 2000 - The book ***Software Cost Estimation with COCOMO II*** (Dr. Barry Boehm, et al) is published to document how to apply the latest estimation model. Most of the original ***Software Engineering Economics*** is still applicable to modern software projects.
- The Center continues to do research on COCOMO (COConstructive COSt MOdel), a tool which allows one to estimate the cost, effort, and schedule associated with a prospective software development project. First published in 1981, the original COCOMO model has recently been superseded by COCOMO II, which reflects the improvements in professional software development practice, positioning COCOMO for continued relevancy into the 21st century.

Empirical Model (COCOMO)

- Provide computational means for deriving S/W cost estimates as functions of variables (major cost drivers)
- Functions used contain constants derived from statistical analysis of data from past projects:
 - can only be used if data from past projects is available
 - must be **calibrated** to reflect local environment
 - relies on initial size and cost factor estimates which themselves may be questionable.

COCOMO

- COCOMO (**C**ONSTRUCTIVE **C**OST **M**ODEL)
 - First published by Dr. Barry Boehm, 1981
- Interactive cost estimation software package that models the cost, effort and schedule for a new software development activity.
 - Can be used on new systems or upgrades
- Derived from statistical regression of data from a base of 63 past projects (2000 - 512,000 DSIs)

COCOMO Mode & Model

- Three development environments (modes)
 - Organic Mode
 - Semidetached Mode
 - Embedded Mode
- Three increasingly complex models
 - Basic Model
 - Intermediate Model
 - Detailed (or Complete) Model

COCOMO Modes

■ Organic Mode

- Developed in familiar, stable environment.
- Relatively small groups working efficiently to develop well-understood applications / domain.
- Routine project, product similar to previously developed product
- Project expected to run smoothly, Typically a smaller system (approx. < 50K DSI)

■ Semi-detached Mode

- Somewhere between Organic and Embedded.
- Project team consists of a mixture of experienced and inexperienced staff.
- Complex system, but something the company is familiar with
- System not huge, but not small either.

Cocomo Modes ...cont'd

- Embedded mode
 - new product requiring a great deal of innovation, difficulties expected
 - Project that is hard (control software for a nuclear plant, or spacecraft)
 - Team has little experience in domain
 - New or inexperienced team
 - Tend to be large projects with lots of Constraints
 - inflexible constraints and interface requirements (ex: strongly coupled to complex hardware, real-time systems)

COCOMO Model (CONT.)

- For each of the three product categories:
 - From size estimation (in KLOC), Boehm provides equations to predict:
 - project duration in months
 - effort in programmer-months
- Boehm obtained these equations:
 - examined historical data collected from a large number of actual projects.
- Software cost estimation is done through three stages:
 - Basic COCOMO,
 - Intermediate COCOMO,
 - Complete COCOMO.

COCOMO Product classes

- Roughly correspond to application, utility and system programs respectively.
 - Data processing and scientific programs are considered to be **application programs**.
 - Classified as Organic
 - Compilers, linkers, editors, etc., are **utility programs**.
 - Classified as Semi-detached
 - Operating systems and real-time system programs, etc. are **system programs**.
 - Classified as Embedded.

Basic COCOMO Model (CONT.)

- Gives only an approximate estimation:
 - $\text{Effort} = a1 (\text{KSLOC})^{a2}$
 - $\text{Tdev} = b1 (\text{Effort})^{b2}$
 - KSLOC is the estimated kilo lines of source code,
 - $a1, a2, b1, b2$ are constants for different categories of software products,
 - Tdev is the estimated time to develop the software in months,
 - Effort estimation is obtained in terms of person months (PMs).

Basic Model

Effort Equation (COCOMO 81)

- $\text{Effort} = A(\text{size})^{\text{exponent}}$
 - A is a constant based on the developmental mode
 - organic = 2.4
 - semi = 3.0
 - embedded = 3.6
 - Size = 1000s Source Lines of Code (KSLOC)
 - Exponent is constant for a given mode
 - organic = 1.05
 - semi = 1.12
 - embedded = 1.20

Development Effort Estimation

□ Organic :

$$\square \text{Effort} = 2.4 (\text{KSLOC})^{1.05} \text{ PM}$$

□ Semi-detached:

$$\square \text{Effort} = 3.0 (\text{KSLOC})^{1.12} \text{ PM}$$

□ Embedded:

$$\square \text{Effort} = 3.6 (\text{KSLOC})^{1.20} \text{ PM}$$

Basic Model

Schedule Equation (COCOMO 81)

- MTDEV (Minimum time to develop) = $2.5 * (\text{Effort})^{\text{exponent}}$
- 2.5 is constant for all modes
- Exponent based on mode
 - organic = 0.38
 - semi = 0.35
 - embedded = 0.32
- Note that MTDEV does not depend on number of people assigned.

Development Time Estimation

- Organic:

- $T_{dev} = 2.5 (\text{Effort})^{0.38} \text{ Months}$

- Semi-detached:

- $T_{dev} = 2.5 (\text{Effort})^{0.35} \text{ Months}$

- Embedded:

- $T_{dev} = 2.5 (\text{Effort})^{0.32} \text{ Months}$

Example

- The size of an organic software product has been estimated to be 32,000 lines of source code.

Solution

□ $\text{Effort} = 2.4 * (32)^{1.05} = 91 \text{ PM}$

□ $\text{Nominal development time} = 2.5 * (91)^{0.38} = 14 \text{ months}$

How to estimate KSLOC?

- Get 2 “experts” to provide estimates.
 - Better if estimates are based on software reqs.
 - Even better if estimates are based on design doc.
 - Good to get best estimate as well as +/- size.
 - Make sure they address integration/glue code/logic.
 - Take average of experts.
- If using Work Breakdown Structure (WBS) in scheduling, estimate KSLOC per task.
 - Note not all “tasks” have KSLOC.
 - Remember COCOMO is strict development effort not management, reporting or user support.
 - COCOMO Does NOT include defining the Requirements /Specification!

Intermediate COCOMO

- Basic COCOMO model assumes effort and development time depend on product size alone.
- However, several parameters affect effort and development time:
 - Reliability requirements, Use of CASE tools, modern facilities to developers, Size of data to be handled
- For accurate estimation, the effect of all relevant parameters must be considered.
- **Intermediate COCOMO model** recognizes this fact:
 - refines the initial estimate obtained by the basic COCOMO by using a set of 15 cost drivers (multipliers).

Intermediate COCOMO (CONT.)

- If modern programming practices are used,
 - initial estimates are scaled downwards.
- If there are stringent reliability requirements on the product :
 - initial estimate may be scaled upwards.
- Rate different cost driver parameters on a scale of one to three, **multiply those values with the estimate obtained using the basic COCOMO.**
- Cost driver classes:
 - Product: Inherent complexity of the product, reliability requirements of the product, etc.
 - Computer: Execution time, storage requirements, etc.
 - Personnel: Experience of personnel, etc.
 - Development Environment: Sophistication of the tools used for software development.

Intermediate COCOMO (CONT.)

Development Effort Estimation

□ Organic :

$$\square \text{Effort} = 3.2 (\text{KSLOC})^{1.05} \text{ PM}$$

□ Semi-detached:

$$\square \text{Effort} = 3.0 (\text{KSLOC})^{1.12} \text{ PM}$$

Same as Basic
Cocomo model

□ Embedded:

$$\square \text{Effort} = 2.8 (\text{KSLOC})^{1.20} \text{ PM}$$

Note the change in coefficients for Organic and Embedded cases in comparison to that for the Basic Cocomo Model.

Intermediate COCOMO (CONT.)

Development Time Estimation

- Organic:

- $T_{dev} = 2.5 (\text{Effort})^{0.38} \text{ Months}$

- Semi-detached:

- $T_{dev} = 2.5 (\text{Effort})^{0.35} \text{ Months}$

- Embedded:

- $T_{dev} = 2.5 (\text{Effort})^{0.32} \text{ Months}$

Note that above equations are same as that for the Basic Cocomo Model

Example

Consider a database system needed for an office automation project.
The requirements document shows 4 modules needed.

Sizes are estimated as follows:

data entry	0.6	KDSI
data update	0.6	KDSI
query	0.8	KDSI
report gen	1.0	KDSI

◦ How can SIZE be estimated in the first place?

From the requirements the ratings were assessed:

The manager rates project details as follows:

characteristic	level	EAF	
complexity	high	1.15	All others: nominal (1.0)
storage	high	1.06	
experience	low	1.13	
programmer capabilities	low	1.17	

Solution

system SIZE 3.0 KDSI

Intermediate
Cocomo

The project is judged to be ORGANIC (so $a=3.2$, $b=1.05$)

Initial effort: $3.2 * 3^{1.05} = 10.14$ PM (person-months)

Person Months for the project:

$$\begin{aligned} \text{PM} &= (1.15 * 1.06 * 1.13 * 1.17) * 3.2 * (3.0)^{1.05} \\ \text{PM} &= 1.61 * 3.2 * 3.17 \\ \text{PM} &= 16.33 \end{aligned}$$

> 16 person months estimated

Now it is an
Adjusted Effort

◦ How many people should we
hire for this effort?

Solution ...cont'd

The model:

Organic	DURATION= 2.5 * EFFORT	0.38
Semi-Detached	DURATION= 2.5 * EFFORT	0.35
Embedded	DURATION= 2.5 * EFFORT	0.32

We had 16.33 person-months estimated for the database project.
The project was judged **ORGANIC**

DURATION = 2.5 * (16.33)
DURATION = 2.5 * 2.89
DURATION = 7.23

> 7 months to complete

How many people to hire?

$$\frac{16.33 \text{ person-months}}{7.23 \text{ months}} = 2.26 \text{ persons}$$

2 or 3 team members needed

Shortcoming of basic and intermediate COCOMO models

□ Both models:

- consider a software product as a single homogeneous entity:
- However, most large systems are made up of several smaller sub-systems.
 - Some sub-systems may be considered as organic type, some may be considered embedded, etc.
 - for some the reliability requirements may be high, and so on.

Complete COCOMO

- ▮ Cost of each sub-system is estimated separately.
- ▮ Costs of the sub-systems are added to obtain total cost.
- ▮ Reduces the margin of error in the final estimate.

Complete COCOMO Example

- A Management Information System (MIS) for an organization having offices at several places across the country:
 - Database part (semi-detached)
 - Graphical User Interface (GUI) part (organic)
 - Communication part (embedded)
- Costs of the components are estimated separately:
 - summed up to give the overall cost of the system.

Example

For a statistical analysis package (size 4.63 KLOC) compute programmer productivity and the development time using Basic Cocomo model.

Average Staff Size

$$SS = \frac{E}{TDEV} = \frac{[\text{staff} - \cancel{\text{months}}]}{\cancel{[\text{months}]}} = [\text{staff}]$$

Productivity

$$P = \frac{Size}{E} = \frac{[\text{KLOC}]}{[\text{staff} - \text{months}]} = \text{KLOC} / \text{staff} - \text{month}$$

Example

let's say Ada programs tend to require 70 DSI per FP.

A system with $FP = 124$

Solution

$$SIZE = FP * 70 = 124 * 70 = 8680$$

SO, the estimate for SIZE in this case is 8.68 KDSI

Productivity comparisons

- Lower level the language, the more productive the programmer
 - The same functionality takes more code to implement in a lower-level language than in a high-level language.
- More verbose the programmer, the higher the productivity
 - Measures of productivity based on lines of code suggest that programmers who write verbose code are more productive than programmers who write compact code.

Productivity estimates

- Real-time embedded systems, 40-160 LOC/P-month.
- Systems programs , 150-400 LOC/P-month.
- Commercial applications, 200-900 LOC/P-month.
- **Later on** - In object points, productivity has been measured between 4 and 50 object points/month depending on tool support and developer capability.